Divide & Conquer

1.

Suppose the company, "Sparkling Ideas," has been new in the market for the past few years and you work as a financial analyst hired for this company.

You have been provided a list of the company's monthly incomes over the last year. However, the incomes are mixed with positive and negative figures, which indicates the company has faced some difficulties during certain months. The company's CFO has a special skill to identify the trend in the company's income details, which helps her make informed decisions. She believes that there is a particular set of consecutive months where the company earns the most, and she wants you to find out which one(s) it is.

Your task is to find the maximum earnings from the given monthly incomes and determine in which part of the year the company earned the most.

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
7	-15	9	-4	12	-8	3	-11	16	-2	6	-10

The figures are in millions of USD.

a) Try to think of an efficient algorithm to solve this problem and provide the result in terms of the starting and ending months, along with the corresponding total income. **Show** a simulation of your algorithm.
b) **Calculate** the time complexity of your algorithm. Show proper mathematical logic.

2.

Imagine you're a farmer named Sam who grows different types of vegetables in several continuous fields. Each field can grow a specific type of vegetable, and based on the market value of that vegetable, each field will bring a certain profit (positive integer) or loss (negative integer). The profit or loss is estimated and noted for each field.

Sam can start farming from any field, but once he starts, he must continue farming the next fields in sequence without skipping any, until he decides to stop, because his tractor can only move to the next adjacent field and cannot skip fields.

Sam needs to choose which sequence of fields to farm to maximize his profits. The following array represents the estimated profits or losses for ten consecutive fields on Sam's farm, based on the types of vegetables that can be grown in each field and their respective market values.

[4, -18, 16, -14, 12, -1, 3, 1, -20, 15]

a) Can you help Sam decide which sequence of fields to farm for maximum profit using an efficient algorithm? What would the profit amount be? **Show** a simulation of your proposed algorithm. You must show which sequence of fields he needs to select in order to achieve this maximum profit.
b) **Calculate** the time complexity of your algorithm using proper mathematical logic. An efficient algorithm should have time complexity less than or equal to $O(N * log(N))$ where N is the number of fields.

3.

Suppose the company, "Sparkling Ideas," has been new in the market for the past few years and you work as a financial analyst hired for this company.

You have been provided a list of the company's monthly incomes over the last year. However, the incomes are mixed with positive and negative figures, which indicates the company has faced some difficulties during certain months. The company's CFO has a special skill to identify the trend in the company's income details, which helps her make informed decisions. She believes that there is a particular set of consecutive months where the company earns the most, and she wants you to find out which one(s) it is.

Your task is to find the maximum earnings from the given monthly incomes and determine in which part of the year the company earned the most.

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
17	-15	9	-4	12	-18	3	11	16	-2	-16	10

The figures are in millions of USD.

c) Try to think of an efficient algorithm to solve this problem and provide the result in terms of the starting and ending months, along with the corresponding total income. **Show** a simulation of your algorithm.
d) **Calculate** the time complexity of your algorithm. Show proper mathematical logic.

4.

i. By showing necessary math, **explain** how Karatsuba's Fast Multiplication algorithm converts an N-digit multiplication to three N/2-digit multiplications.
ii. Can we modify the algorithm to multiply two N-bit binary numbers? **Explain** how or why not.

5.

Your friend gave you a binary string B (meaning each character is either **0** or **1**). He wanted to find out how to calculate the maximum number of consecutive 0s in that particular string. For example,

String: 100100000111	Maximum consecutive 0s: 5
String: 1010101010101	Maximum consecutive 0s: 1

You, as an algorithm enthusiast, know that this can be solved in linear time. However, your friend asked you to propose a Divide and Conquer approach

a) Name a suitable Divide and Conquer algorithm for this task.
b) Explain how you can apply that algorithm in this scenario. Present your idea in a pseudocode/programmable code/Flowchart/step-by-step instructions.
c) Write the time complexity of your algorithm.

6.

John is an amateur poker player who has decided to try his luck at a local casino. The casino allows players to play in rounds, each leading to a win (positive integer) or a loss (negative integer). John keeps track of his net gain or loss after each round.

John knows that poker isn't just about luck—it's about making strategic decisions. And one of the critical decisions he has to make is knowing when to stop playing. However, the casino has a rule that once a player starts playing rounds, they must play the subsequent rounds in order without skipping any, until they decide to stop, as skipping rounds might disrupt the flow of the game and disadvantage other players.

With this in mind, John wants to find the optimal sequence of rounds where he can maximize his net winnings. The following array represents the net gain or loss John experiences in ten rounds of poker at the casino.

[8, -18, -8, 2, 16, -14, 2, -1, 3, 15]

c) Find an efficient algorithm that will help John to figure out which sequence of rounds he should play to maximize his winnings. What would be the exact profit amount? **Show** a simulation of your proposed algorithm. You must show which sequence of rounds he needs to select in order to achieve this maximum profit.
d) **Calculate** the time complexity of your algorithm using proper mathematical logic. An efficient algorithm should have time complexity less than or equal to $O(N * log(N))$ where N is the number of fields.

7.

Inspired by Karatsuba's algorithm for multiplying integers, your friend has come up with the following divide-and-conquer algorithm for squaring n-digit numbers. Pseudocode for the algorithm is given below. Your job is to fill in the details. You may assume that n is a power of two.

Algorithm 1: KARAT-SQUARE(x)

```
   Input  : An n-digit number x
   Output: x^2
1  if n = 1 then
2  |  return x^2
3  end
4  a ← the leftmost n/2 digits of x;
5  b ← the rightmost n/2 digits of x;
6  S1 ← KARAT-SQUARE(a) ;                          /* S1 equals a^2 */
7  S2 ← KARAT-SQUARE(b) ;                          /* S2 equals b^2 */
8  S3 ← KARAT-SQUARE (_____) ;                     /* fill in the blank */
9  P ← _____ ;                                     /* P should equal 2ab */
10 return _____ × 10^n + P × 10^(n/2) + _____;
```

Now carefully read the algorithm above, and answer the following questions.

i) Write x in terms of a and b. Looking at lines 4 and 5 of the algorithm should help.

ii) Square your answer from (i). The formula $(m + n)^2 = m^2 + 2mn + n^2$ should help.

iii) Fill in the blank in line 8. Your answer should involve a and b in some way. The idea is to compute a square that, along with a^2 and b^2, helps you compute $2ab$. You may want to think about questions (iii) and (iv) at the same time.

iv) Fill in the blank in line 9. Your answer should involve S_1, S_2, and S_3 in such a way that P equals $2ab$.

v) Fill in the blank in line 10.

vi) Write the running time of your algorithm above.

vii) Say you have another algorithm KARAT-SQUARE-2(x) that takes an n-digit number and squares it by recursively squaring six $n/3$-digit numbers, and combining them in $O(n)$ time. Which algorithm is asymptotically faster: KARAT-SQUARE(x) or KARAT-SQUARE-2(x)?

8.

i. By showing necessary math, **explain** how Karatsuba's Fast Multiplication algorithm converts an N-digit multiplication to three N/2-digit multiplications.

ii. Can we modify the algorithm to multiply two N-digit hexadecimal numbers? **Explain** how or why not.

9.

A football team's analytics department hired you and gave you a task of reviewing match performance data from the past season. Each match's net goal difference (goals scored minus goals conceded) is recorded in a list. Now you were asked by the department to identify the team's best consecutive performance streak, where the net goal difference was maximized over a series of matches.

net_goal_difference: [1, 4, 3, -5 , 5, 6, 1, -4]

To prove your skills, you used an approach where the problem is divided into subproblems to find the solution. Now tell us more about it by answering the following questions.

i. For the given input above, **determine** the number of times the *Cross-Sum* exceeds both the *Left-Sum* and the *Right-Sum*. Exclude base cases (subarrays of size 1) from consideration. Show your work properly.
Cross-Sum: sum of elements calculated by combining sum from mid to left and mid to right of the current subarray.
Left-Sum: maximum sum in the left half of the subarray.
Right-Sum: maximum sum in the right half of the subarray.

ii. The department wants you to find the maximum consecutive matches with a positive goal difference. Now **present** your idea to solve the problem considering you have to use the same approach used earlier with pseudocode/programmable code/step-by-step logical instructions.

10.

A stock trading firm hired you to analyze daily stock price changes of a company over the past quarter. Each day's net price change (price at the end of the day minus price at the start) is recorded in a list. You were asked by the firm to identify the company's best consecutive stock price increase, where the price change was maximized over a series of days.

Price_changes = [2, 3, 5, -7, 3, 2, 1, -6]

To prove your skills, you used an approach where the problem is divided into subproblems to find the solution. Now tell us more about it by answering the following questions.

i. For the given input above, **determine** the number of times the *Cross-Sum* exceeds both the *Left-Sum* and the *Right-Sum*. Exclude base cases (subarrays of size 1) from consideration. Show your work properly.
Cross-Sum: sum of elements calculated by combining sum from mid to left and mid to right of the current subarray.
Left-Sum: maximum sum in the left half of the subarray.
Right-Sum: maximum sum in the right half of the subarray.

ii. The firm wants you to find the maximum consecutive days with a positive price change. Now **present** your idea to solve the problem considering you have to use the same approach used earlier with pseudocode/programmable code/step-by-step logical instructions.

11.

Imagine yourself helping Taylor Swift following her "Eras Tour first show." Taylor is curious about her performance and the groups of songs her audience connected most with them. She receives comments for every song on her setlist: some calm the audience down a bit (***and have a negative score***), while others get the crowd quite thrilled (***and have a positive score***).

Taylor can choose any song to start her performance, but she likes to keep the flow in order. So the next song she sings has to be the one right after she has chosen the starting song on the list, regardless of the public reactions**.**

Given a list of scores for each song (indicating how much they boost or calm the crowd), can you:

[-3, 7, 12, -8, -2, 83, -7, 4]

i. Help Taylor determine which segment of consecutive songs she should perform at her next concerts to maximize audience interest. What is the total comment score of this segment? **Simulate** with proper steps.

ii. **Explain** the time taken by your preferred method, particularly given the *"n"* numbers of songs Taylor has in her setlist. Ideally, your method should not exceed $O(nlgn)$.

12.

Imagine you're helping Dr. Eleanor, who's studying old papers about a long-gone city. Each paper talks about a year in the city's life. Some events brought prosperity to the civilization (***represented by a positive number***), while others brought decline and challenges (***represented by a negative number***).

Dr. Eleanor believes it is essential to study events in a continuous sequence to fully understand the rise and fall of civilizations. She can start with any manuscript, but once she begins, she must analyze the remaining manuscripts sequentially to maintain the timeline,

Given a list of numbers for each manuscript (indicating the prosperity or decline they brought), can you:

[2, -6, 8, 8, -9, -1, -2, 20]

a. Tell Dr. Eleanor which group of years she should read about to learn of the city's best times. How good were those years combined? Show in detail how you found this out.
b. Explain the time taken by your preferred method, particularly given the "*n*" numbers of manuscripts Dr. Eleanor has in her studies. It should be an easy and quick method, as she has many papers to go through!

13.

3 In Rivendell, **n** voters are casting their votes for various political parties. The votes are recorded in an array, where each number represents a party's ID.

However, Rivendell follows a special rule: a party can only form the government if it receives more than $\lfloor \frac{n}{2} \rfloor$, that is, a strict majority. Otherwise, the election results in a deadlock, and a re-election must be held. For example, in the array [1,2,2,2,3,2,1], party 2 wins the majority since it got 4 votes (more than $\lfloor \frac{7}{2} \rfloor = 3$).

a. **CO1** Can two different parties each secure a majority in any given vote array? **Explain** why or why not. **02**

b. **CO1** Given a vote array and a party ID, we need to determine whether the party has won the majority in the array. **Propose** an O(N) algorithm to verify this, where N is the number of voters. **02**

c. **CO1** Now, we want to apply a divide and conquer approach to find the majority from a given vote array. Like merge sort, if we divide the vote array into two halves, can the majority element in each half be different from the majority of the entire array? **Justify** your answer with an example. **02**

d. **CO1** Reuse your algorithm of Q3(b) to complete the following algorithm to find the majority party from the vote array. The algorithm should return -1 if there's no majority party. You can use either pseudo-code or step-by-step instructions. **04**

```
function majority(arr):
      if length(arr) == 1:
            return arr[0]

      mid = length(arr) // 2
      left_majority = majority(arr[0 ... mid])
      right_majority = majority(arr[mid+1 ... length(arr)-1])

      # Merge Step: Reuse your algorithm of Q3(b) to complete
```

14.

You are participating in a competitive multiplayer strategy game where the objective is to defeat your opponent within a fixed time constraint. Throughout the game, various perks are provided at each level in the form of health boxes. At **Level 1**, you are presented with a list of health boxes, each represented by an integer. Positive values indicate a health boost to your own character, while negative values correspond to a reduction in your opponent's health. At this level, you are allowed to select exactly one box, and you want to maximize the health damage to your opponent. This list of health points is given below:

```
[2, 3, -4, -3, 5, -1, 2, -2]
```

Here, each element corresponds to a distinct health box.

a **CO1** For the above scenario, can you find the box that will cause the maximum possible reduction in enemy health by applying a **divide and conquer** approach? **Explain** your idea in brief, *no need to write full code*. **02**

b **CO3** At **Level 2**, you are offered the same collection of health boxes. However, this time you are allowed to choose **one or a block of consecutive boxes without gaps**. Now you want to increase your own health by collecting points that yield the highest points gain. You came up with an algorithm with **O(N*logN)** time complexity to compute this optimal health gain. Where 'N' is the number of health boxes. **03**

Show a simulation of the algorithm with proper steps.

15.

You are participating in a competitive multiplayer strategy game where the objective is to defeat your opponent within a fixed time constraint. Throughout the game, various perks are provided at each level in the form of health boxes. At **Level 1**, you are presented with a list of health boxes, each represented by an integer. Positive values indicate a health boost to your own character, while negative values correspond to a reduction in your opponent's health. At this level, you are allowed to select exactly one box, and you want to maximize the health damage to your opponent. This list of health points is given below:

```
[-2, 3, 4, -3, -5, 1, 2, 2]
```

Here, each element corresponds to a distinct health box.

a **CO1**	For the above scenario, can you find the box that will cause the maximum possible reduction in enemy health by applying a **divide and conquer** approach? **Explain** your idea in brief, *no need to write full code*.	**02**

b **CO3**	At **Level 2**, you are offered the same collection of health boxes. However, this time you are allowed to choose **one or a block of consecutive boxes without gaps**. Now you want to increase your own health by collecting points that yield the highest points gain. You came up with an algorithm with **O(N*logN)** time complexity to compute this optimal health gain. Where 'N' is the number of health boxes. **Show** a simulation of the algorithm with proper steps.	**03**

16.

We are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example $A: \{15, -13, 3, -4, 5, -6\}$.

We want to find the **Max-Prod Subarray** of the input. Just like the Max-sum Subarray, but instead of summation, we have to consider the product (multiplication) of the numbers.

We have come up with a divide-and-conquer algorithm to solve this problem. This algorithm has the same concept that you know for the Max-sum Subarray, just modified for multiplication.

```
function MaxProdSubarray(A, left, right){
    if (left == right):
        return A[left]

    mid = (left + right) / 2

    LPS = MaxProdSubarray(A, left, mid)
    RPS = MaxProdSubarray(A, mid + 1, right)
    CPS = CrossProduct(A, left, mid, right)

    return max(LPS, RPS, CPS)
}

function CrossProduct(A, left, mid, right){
    leftProd = -∞
    prod = 1
    for (i from mid to left){
        prod = prod * A[i]
        if (prod > leftProd): leftProd = prod
    }

    rightProd = -∞
    prod = 1
    for (i from mid+1 to right){
        prod = prod * A[i]
        if (prod > rightProd): rightProd = prod
    }
    return leftProd * rightProd
}
```

a. **CO1** **Mark** the changes that you see in this algorithm than the one for Max-Sum Subarray. You can either write only those lines in your script. Or, write the whole code given above and underline the ones you want to mark. Also, you don't have to mark the changes in variable and function names. **02**

b. **CO3** **Evaluate** whether the given algorithm would actually work for this problem. If your answer is yes, give a proper logical explanation. Otherwise, give an input where the algorithm will not produce the correct answer. **03**

c. **CO2** We are thinking, why not use Karatsuba's multiplication technique to make our algorithm faster? So we replace the multiplication operations (*) with Karatsuba's function. For example, **02**

$prod * A[i]$ becomes $Karatsuba(prod, A[i])$.

Write a recurrence relation [$T(N) =$ ___ + ___] to express the time complexity of the algorithm after applying Karatsuba. Assume that Karatsuba's time complexity is $O(N^{1.58})$ (same N as the size of the Array A).

d. **CO2** Now **solve** your recurrence from Q3(c) to find the actual time complexity. Any method is acceptable as long as you show proper work. **02**